

Indian Institute of Technology Madras
Department of Data Science and Artificial Intelligence

DA2304: Introduction to Computer Systems

Assignment 3: Jack Compiler & 2D Convolution

Name: *Mohammed Khaja Ghouse Mohiuddin*
Roll No.: *DA24B015*
Instructor: Dr. Patanjali SLPSK
Deadline: May 20, 2026

1 Part 1: Jack Convolution Program

1.1 Design Decisions

1.1.1 *Kernel Choice: Laplacian Edge Detector*

The 3×3 filter used is the discrete Laplacian:

$$K = \begin{bmatrix} -1 & -1 & -1 \\ -1 & 8 & -1 \\ -1 & -1 & -1 \end{bmatrix}$$

This is a popular filter that computes a weighted difference between each pixel and its 8 neighbours. In a uniformly increasing gradient, interior pixels with equal spacing in every direction will produce near-zero outputs, while edge and corner pixels where the symmetry breaks produce large magnitudes. This makes it ideal for a quick visual sanity check.

1.1.2 *Stride & Padding*

We set `stride = 1` and `P = 0` (no padding). For a 5×5 input the output dimension is:

$$O = \left\lfloor \frac{5 - 3 + 0}{1} \right\rfloor + 1 = 3$$

1.1.3 *Additional Fields and Design Choices*

One field was added beyond the skeleton:

- `field int outputDim`: this caches the computed output side-length so that `conv2d()` and `printOutput()` do not recompute it. This helps avoids two redundant integer divisions per call.

1.1.4 *Memory Layout*

All matrices are stored as flattened 1D arrays in row-major order. Element (i, j) of an $N \times N$ matrix is at index $i \cdot N + j$. `ArrayFilter` is `static` (shared across all instances); `ArrayInput`, `ArrayOutput`, and `outputDim` are `field` (per-instance).

1.2 Test Case 1: Linear Gradient (5×5 , stride=1)

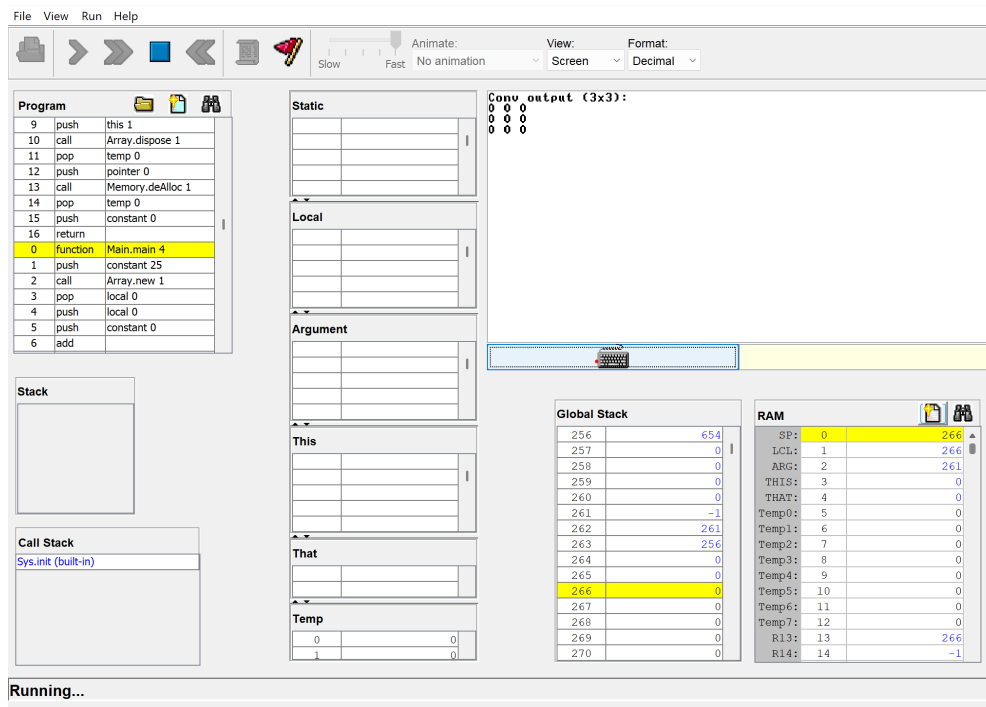
1.2.1 Input & Expected Output

The input is the integers 1 through 25 in row-major order:

$$\text{Input} = \begin{bmatrix} 1 & 2 & 3 & 4 & 5 \\ 6 & 7 & 8 & 9 & 10 \\ 11 & 12 & 13 & 14 & 15 \\ 16 & 17 & 18 & 19 & 20 \\ 21 & 22 & 23 & 24 & 25 \end{bmatrix} \implies Y = \begin{bmatrix} 0 & 0 & 0 \\ 0 & 0 & 0 \\ 0 & 0 & 0 \end{bmatrix}$$

All zeros: the Laplacian of a perfect linear gradient is identically zero for every interior window position, since the positive centre weight (+8) exactly cancels the sum of the eight neighbours (−1 each). Any non-zero entry would signal a bug in the convolution logic.

1.2.2 VM Emulator Screenshot



1.3 Test Case 2: Impulse Response (5×5 , stride=1)

To further validate correctness, a second test was run with a single bright pixel (value 50) at the centre of an otherwise zero input. This computes the filter's *impulse response* ie., the output directly reveals each filter weight scaled by the impulse magnitude.

1.3.1 Input & Expected Output

$$\text{Input} = \begin{bmatrix} 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 50 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \end{bmatrix} \implies Y = \begin{bmatrix} -50 & -50 & -50 \\ -50 & 400 & -50 \\ -50 & -50 & -50 \end{bmatrix}$$

The centre output element is $8 \times 50 = 400$ (the impulse lands on the +8 filter weight). All surrounding elements are $-1 \times 50 = -50$ (the impulse lands on a -1 weight in each case). This confirms that every filter coefficient is applied at the correct spatial position.

1.3.2 VM Emulator Screenshot

The screenshot displays a VM Emulator interface with the following components:

- Program:** A list of instructions with line numbers and labels.

343	pop	temp 0
344	push	local 2
345	call	Conv.printOutput 1
346	pop	temp 0
347	push	local 2
348	call	Conv.dispose 1
349	pop	temp 0
350	push	local 0
351	call	Array.dispose 1
352	pop	temp 0
353	push	local 1
354	call	Array.dispose 1
355	pop	temp 0
356	push	constant 0
357	return	
- Static:** A table showing static variables.

0	0
1	0
2	0
3	0
4	0
- Local:** A table showing local variables.

0	0
1	0
2	0
3	0
4	0
- Argument:** A table showing arguments.

0	0
1	0
2	0
3	0
4	0
- This:** A table showing 'This' pointers.

- That:** A table showing 'That' pointers.

- Temp:** A table showing temporary variables.

0	0
1	0
- Conv output (3x3):** A 3x3 matrix of values.

-50	-50	-50
-50	400	-50
-50	-50	-50
- Global Stack:** A table showing global stack memory.

256	0
257	0
258	0
259	0
260	0
261	0
262	0
263	0
264	0
265	0
266	0
267	0
268	0
269	0
270	0
- RAM:** A table showing RAM memory.

SP:	0	256
LCL:	1	0
ARG:	2	0
THIS:	3	0
THAT:	4	0
Temp0:	5	0
Temp1:	6	0
Temp2:	7	0
Temp3:	8	0
Temp4:	9	0
Temp5:	10	0
Temp6:	11	0
Temp7:	12	0
R13:	13	0
R14:	14	0

Running...

2 Part 2: Jack Compiler Frontend

2.1 Architecture

The compiler consists of five Python modules:

Module	Responsibility
<code>JackTokenizer.py</code>	Lexical analysis: strips comments via a character-level state machine, classifies tokens into <code>keyword</code> , <code>symbol</code> , <code>integerConstant</code> , <code>stringConstant</code> , or <code>identifier</code> . Emits <code>*T.xml</code> .
<code>SymbolTable.py</code>	Two-scope symbol table. Class scope holds <code>static</code> and <code>field</code> variables; subroutine scope holds <code>argument</code> and <code>var</code> (local) variables. Each symbol stores <code>name</code> , <code>type</code> , <code>kind</code> , and a running <code>index</code> within its kind. Subroutine scope is reset at each new method/function/constructor.
<code>VMWriter.py</code>	Thin abstraction that writes <code>push</code> , <code>pop</code> , <code>arithmetic</code> , <code>label</code> , <code>goto</code> , <code>if-goto</code> , <code>function</code> , <code>call</code> , and <code>return</code> to a <code>.vm</code> file.
<code>CompilationEngine.py</code>	Recursive-descent parser. Implements every non-terminal in the Jack grammar. Simultaneously emits the parse-tree XML (<code>*.xml</code>) and VM code (<code>*.vm</code>) via the <code>VMWriter</code> .
<code>JackCompiler.py</code>	Entry point. Accepts a <code>.jack</code> file or directory, runs the tokenizer then the compilation engine on each file, and writes all output to the <code>out/</code> folder.

3 Exercise 3: Analysis Questions

3.1 Q1: Why the Tokenizer Needs a State Machine for Block Comments

A single-line regex such as `/\/*.*?\*/` applied per line will fail on any comment that spans multiple lines. Consider:

Listing 1: Multi-line comment failure case

```
1 /* This comment
2    spans two lines */
3 var int x;
```

A per-line regex sees line 1 as starting a comment that never closes (no `*/` on that line), so it either strips the entire line and leaves line 2 intact (including the closing `*/`, which then appears as stray code), or fails to strip anything. In either case the subsequent tokens are corrupted.

The state-machine approach tracks a boolean `in_block_comment` flag across line boundaries: when `/*` is encountered the flag goes true and all characters are skipped until `*/` is found, regardless of how many newlines intervene. Additionally, the state machine must track `in_string` to avoid stripping `/*` sequences that appear inside string literals.

3.2 Q2: Symbol Table State Inside `Conv.conv2d()`

When the parser enters `conv2d()`, the symbol table contains:

3.2.1 Class Scope (*this persists for entire class*)

Name	Type	Kind	Index
ArrayFilter	Array	static	0
stride	int	static	1
ArrayInput	Array	field	0
ArrayOutput	Array	field	1
ArraySize	int	field	2
outputDim	int	field	3

3.2.2 Subroutine Scope (*this resets at conv2d entry*)

`conv2d` is a method, so `this` (type `Conv`) is implicitly argument 0. The declared local variables are:

Name	Type	Kind	Index
this	Conv	argument	0
outRow	int	var	0
outCol	int	var	1
fRow	int	var	2
fCol	int	var	3
inRow	int	var	4
inCol	int	var	5
sum	int	var	6
k	int	var	7

At the point where `sum` is assigned (the MAC accumulation line), all 6 class-scope and 9 subroutine-scope identifiers are in scope. The lookup order is subroutine-first, so `sum` resolves to `local 6` and `ArrayInput` resolves to `this 0`.

3.3 Q3: Tracing `let ArrayOutput[k] = sum;`

The compiled VM code for this line is:

Listing 2: VM code for array-indexed assignment

```

1  push this 1          // push base address of ArrayOutput (field
    1)
2  push local 7         // push k
3  add                  // base + k = target address
4  push local 6         // push sum (the value to store)
5  pop temp 0           // save RHS value in temp 0
6  pop pointer 1        // set THAT = target address
7  push temp 0          // recover the RHS value
8  pop that 0           // RAM[THAT + 0] = sum --> ArrayOutput[k]
    ] = sum

```

Step-by-step:

1. `push this 1` pushes the *base address* stored in `RAM[THIS + 1]`, which is the pointer to the heap block that `Array.new` returned for `ArrayOutput`.
2. `push local 7`; `add` computes `base + k`, yielding the exact RAM address of `ArrayOutput[k]`.
3. The RHS expression (`sum`) is compiled next and lands on the stack. At this point the stack has `[targetAddr, sumValue]`.
4. `pop temp 0` saves the value; `pop pointer 1` writes the target address into the `THAT` register (`RAM[4]`).
5. `push temp 0`; `pop that 0` writes `sum` into `RAM[THAT + 0]`, which is exactly `ArrayOutput[k]`.

This two-step juggle via `temp 0` is necessary because both the target address and the value are on the stack, and `pointer 1` can only be set by popping.

3.4 Q4: Convolution Stride Analysis

For a 9×9 input with $F = 3$ and $S = 2$:

$$O = \left\lfloor \frac{9 - 3 + 0}{2} \right\rfloor + 1 = \lfloor 3 \rfloor + 1 = 4$$

The output is $4 \times 4 = 16$ elements.

Does stride change the asymptotic VM-instruction count?

Yes. The dominant cost of `conv2d` is proportional to $O^2 \times F^2$, where O depends on S . Since $O = \Theta\left(\frac{N}{S}\right)$ for fixed F , the total work is:

$$O^2 \cdot F^2 = \Theta\left(\frac{N^2}{S^2}\right) \cdot F^2$$

Doubling the stride from 1 to 2 reduces the number of output positions by a factor of 4 (from $O_1^2 = 49$ to $O_2^2 = 16$ for $N = 9$), and therefore reduces the total VM instruction count by roughly $4\times$. However, each individual MAC (the inner F^2 loop) performs the same number of multiply-accumulates regardless of stride, so the *per-element* cost is unchanged. Only the number of elements changes.

In asymptotic terms, $S = 1$ is $\Theta(N^2 F^2)$ and $S = 2$ is $\Theta(N^2 F^2/4)$ which is the same Θ class but a constant-factor reduction in practice.

Extra Credit

an interesting bug that I encountered:

During initial testing, the parser crashed on `conv2d()` at the line `let ArrayOutput[k] = sum;`. The error was Expected `'('` got `'('`, which was very confusing at first.

The root cause was in `_compile_subroutine_call`: when handling a bare `name(...)` call (a method invocation on `this`), the original code consumed an extra token with `_process()`, thinking it needed to handle the already-consumed function name. This accidentally ate the opening parenthesis, so when the code subsequently tried to `_process("(")`, it found the closing parenthesis instead.

The fix was simple, i just had to remove the spurious `_process()` call, since the identifier had already been consumed by the caller (`compile_term` or `compile_do`). The debugging process involved printing the token stream position at each `_process` call and comparing the expected vs. actual token, which immediately revealed the off-by-one consumption.